DEVELOPER SETUP HANDBOOK

Install · Configure · Verify · Troubleshoot

Basic Definitions for Every Core Environment Skill

A quick-reference glossary of the tools, commands, and concepts a practical developer should be able to install, verify, and fix.

How to use this document

This handbook lists the core setup skills a working developer is expected to handle, and pairs each one with a short, plain-language definition. It is written as a glossary you can scan, not a step-by-step tutorial: the goal is that when a term or command appears in an error log, a README, or a teammate's message, you already know what it means and why it exists.

Terms are grouped into fifteen areas, from operating-system setup through troubleshooting. Commands are shown in monospace and can be copied directly into a terminal.

1. Operating System Setup

Getting a working, updated OS onto a machine and understanding how it boots.

Windows installation — Installing Microsoft Windows from a bootable USB or disc, including choosing a drive and completing first-time setup.

Ubuntu / Linux installation — Installing a Linux distribution (Ubuntu is the common default) from a bootable USB, popular for development and servers.

Dual boot — Installing two operating systems on one machine and choosing which to start at boot time.

Clean installation — Wiping the target drive and installing the OS fresh, with no leftover files or settings from a previous system.

Disk partitioning — Dividing a physical drive into separate logical sections (partitions), each of which can hold an OS or data.

BIOS / UEFI — Low-level firmware that starts before the OS. It controls boot order and hardware settings; UEFI is the modern replacement for the older BIOS.

System update — Downloading and applying the latest patches and packages right after install to fix bugs and security holes.

Terminal / command line — A text-based interface for typing commands to control the computer, often faster and more precise than the graphical UI.

2. GPU, CUDA, and AI Development Setup

Making an NVIDIA GPU usable for AI/ML work, and confirming the software stack can see it.

NVIDIA driver — The software that lets the operating system talk to an NVIDIA GPU. Without it, the GPU is not usable.

CUDA Toolkit — NVIDIA's platform and libraries that let programs run general-purpose computation on the GPU.

cuDNN — An NVIDIA library of GPU-accelerated building blocks for deep learning, used on top of CUDA by frameworks like PyTorch and TensorFlow.

Driver vs. CUDA Toolkit vs. PyTorch CUDA — The driver talks to the hardware; the CUDA Toolkit is the system-level compute platform; the PyTorch CUDA version is the CUDA build bundled inside PyTorch. They are related but versioned separately, and mismatches are a common source of errors.

GPU memory (VRAM) — The dedicated memory on the GPU. Models and data must fit in it; running out causes out-of-memory errors.

Clearing / offloading GPU memory — Freeing VRAM by releasing tensors, emptying the cache, or moving data to CPU/disk so new work can run.

Multi-GPU — Using more than one GPU together to train or serve larger models faster.

CUDA_VISIBLE_DEVICES — An environment variable that tells a program which GPU(s) it is allowed to use, e.g. to pin work to a single card.

KEY COMMANDS

```
nvidia-smi
```

Shows the driver version, each GPU, and its live memory and utilization.

```
nvcc --version
```

Prints the installed CUDA Toolkit compiler version.

```
torch.cuda.is_available()
```

Returns True if PyTorch can see and use a GPU.

```
torch.cuda.get_device_name(0)
```

Prints the name of the first visible GPU.

3. Python Installation and Environment Management

Installing Python and keeping each project's dependencies isolated and reproducible.

Python installation — Adding the Python interpreter to the system so you can run Python programs.

System Python vs. base vs. dedicated environment — System Python ships with the OS and should be left alone; the base environment is the default install; a dedicated environment is a per-project sandbox that keeps its packages separate.

Virtual environment (venv) — An isolated Python setup for one project, so its packages don't clash with other projects.

Conda environment — An isolated environment managed by Conda, which can handle Python and non-Python dependencies together.

Activate / deactivate — Turning an environment on (so its Python and packages are used) or off (returning to the default).

requirements.txt — A text file listing a project's exact package dependencies so they can be reinstalled anywhere.

Dependency conflict — When two packages require incompatible versions of a shared dependency; isolation and pinned versions help avoid this.

KEY COMMANDS

```
python --version
```

Checks which Python version is active.

```
python -m venv env
```

Creates a virtual environment in a folder named env.

```
pip install -r requirements.txt
```

Installs every dependency listed in the file.

```
pip freeze > requirements.txt
```

Saves the currently installed packages and versions to the file.

4. AI / ML Dependencies Setup

Installing the machine-learning libraries and models a project needs, with the right GPU support.

PyTorch with CUDA support — Installing the PyTorch build that matches your CUDA version so training and inference run on the GPU.

TensorFlow — An alternative deep-learning framework, installed when a project requires it instead of (or alongside) PyTorch.

Common ML libraries — Transformers and Hugging Face (models and tooling), Accelerate (multi-device training), OpenCV (image/video), NumPy and Pandas (numeric and tabular data), FastAPI (serving) — the everyday toolkit.

Model-specific dependencies — Extra packages a particular model needs beyond the standard stack, usually listed in its README.

Large model downloads — Fetching model weight files, which can be many gigabytes and need enough disk space and a stable connection.

Hugging Face cache — A local folder where downloaded models are stored so they aren't re-downloaded each run; its location can be changed with HF_HOME.

Model loading / inference testing — Loading a model into memory and running a small sample input to confirm it produces sensible output before building on it.

5. Docker and Deployment Basics

Packaging an application with its dependencies so it runs the same way on any machine.

Docker — A tool that packages an app and everything it needs into a container that runs identically across environments.

Dockerfile — A recipe file of instructions describing how to build a container image.

Image — A built, read-only template created from a Dockerfile; containers are run from images.

Container — A running instance of an image — an isolated, lightweight process with its own filesystem.

Exposing ports — Mapping a port inside the container to a port on the host so the app is reachable from outside.

Docker Compose — A tool for defining and running multi-container apps from a single YAML file.

Volume mounting — Linking a host folder into a container so data persists and can be shared.

Environment variables in containers — Passing configuration and secrets into a container at run time instead of hard-coding them.

Docker logs — The output a container produces, used to see what it's doing and to debug failures.

NVIDIA Container Toolkit — The add-on that lets Docker containers access the host GPU for accelerated workloads.

6. Git and GitHub Management

Tracking code changes over time and collaborating through a shared remote repository.

Git — A version-control system that records changes to files so you can review history and collaborate.

GitHub — A hosting service for Git repositories that adds collaboration features like pull requests and issues.

Repository (repo) — A project's folder tracked by Git, including all its files and complete change history.

Clone — Downloading a full copy of a remote repository to your machine.

Branch — A separate line of development that lets you work on changes without affecting the main code.

Commit — A saved snapshot of your changes with a descriptive message.

Push / pull — Push uploads your commits to the remote; pull downloads and merges others' commits.

Pull request (PR) — A proposal to merge one branch into another, opened for review before it is accepted.

Merge conflict — When two changes touch the same lines and Git can't combine them automatically, requiring manual resolution.

.gitignore — A file listing paths Git should ignore, such as secrets, build output, and virtual environments.

SSH key — A cryptographic key pair that authenticates you to GitHub without typing a password each time.

Personal access token (PAT) — A generated secret used in place of a password for HTTPS Git operations and APIs.

main vs. master — The default primary branch name; newer repos use main, older ones use master.

KEY COMMANDS

```
git config --global user.name "Your Name"
```

Sets the author name attached to your commits.

```
git config --global user.email "you@email.com"
```

Sets the author email attached to your commits.

7. FFmpeg and Media Tools

A command-line toolkit for converting and processing audio and video, common in AI media pipelines.

FFmpeg — A powerful command-line tool for recording, converting, and streaming audio and video.

Adding FFmpeg to PATH — Registering FFmpeg's location so it can be run from any terminal by name.

Audio/video conversion — Changing a media file from one format or codec to another.

Extracting audio — Pulling the audio track out of a video into a separate file.

Splitting and merging — Cutting media into segments or joining several files into one.

Use in AI pipelines — Preparing media for tasks like speech-to-text (STT), dubbing, and general video processing.

KEY COMMANDS

```
ffmpeg -version
```

Confirms FFmpeg is installed and shows its version.

8. Microsoft Visual C++ / Build Tools

The compilers and libraries some Python packages need to build native code during installation.

Visual C++ Redistributable — Runtime libraries many Windows programs depend on; missing them causes DLL errors.

C++ Build Tools — The compiler toolchain on Windows needed to build packages that include C/C++ code.

Why packages need build tools — Some Python packages ship source that must be compiled locally; without a compiler, installation fails.

cmake / ninja — Build-system tools that configure and drive compilation of native projects.

Linux equivalents — On Linux, build-essential provides gcc, g++, and make — the standard compiler toolchain.

9. Environment Variables

Named values the system and programs read at run time to configure behavior and hold secrets.

Environment variable — A key–value setting available to programs, used for configuration and secrets.

PATH — The list of folders the shell searches to find executable commands.

PYTHONPATH — Extra folders Python searches when importing modules.

CUDA_HOME — Points to the CUDA Toolkit install location so tools can find it.

HF_HOME — Sets where Hugging Face stores its cache and downloaded models.

CUDA_VISIBLE_DEVICES — Restricts which GPUs a program can see and use.

.env file — A file holding environment variables (often API keys) that an app loads at startup, kept out of version control.

10. Development Tools Installation

Editors and IDEs, plus the extensions and settings that make day-to-day coding smoother.

VS Code — A lightweight, extensible code editor widely used for Python, web, and general development.

Cursor — An AI-assisted code editor built on VS Code's foundation.

PyCharm — A full-featured Python IDE with strong debugging and project tooling.

Useful extensions — Add-ons for Python, Docker, Git, Jupyter, and Remote SSH that extend the editor's capabilities.

Integrated terminal — A terminal built into the editor so you can run commands without switching windows.

Debugging in the IDE — Running code with breakpoints to pause execution and inspect variables step by step.

Formatting and linting — Tools that auto-format code and flag style or error issues to keep code clean and consistent.

11. Remote Server / GPU Machine Usage

Connecting to and operating a machine you don't sit in front of, and keeping work running on it.

SSH login — Securely connecting to a remote machine's command line over the network.

SCP / SFTP — Protocols for copying files securely between your machine and a remote server.

tmux / screen — Terminal multiplexers that keep long-running processes alive after you disconnect.

Resource monitoring — Checking CPU, RAM, disk, and GPU usage to see how loaded a machine is.

Killing stuck processes — Ending a process that has hung or is consuming resources it shouldn't.

Checking open ports — Seeing which network ports are listening, useful for reaching or debugging services.

Running / accessing remote APIs — Serving an API on the remote machine and reaching it from a browser or Postman, often via an SSH tunnel or exposed port.

12. API Testing and Application Testing

Running a web service locally or remotely and confirming its endpoints behave as expected.

FastAPI / Flask — Python frameworks for building web APIs; FastAPI is modern and async-friendly, Flask is minimal and classic.

Postman / curl — Tools for sending test requests to an API — Postman is a GUI, curl is a command-line tool.

Health check endpoint — A simple endpoint (often /health) that returns OK to confirm a service is up.

Logs — The running record of what an application is doing, the first place to look when something breaks.

File upload / inference APIs — Endpoints that accept uploaded files or run a model on input and return a prediction.

Ports, localhost, and server IPs — A port identifies a service on a machine; localhost (127.0.0.1) is your own machine; a server IP addresses a remote one.

13. Basic Linux Commands

The everyday commands for moving around, managing files and processes, and inspecting a system.

Navigation — Moving through and inspecting the filesystem.

Permissions — Controlling who can read, write, or execute a file, and who owns it.

Process management — Listing running processes and stopping ones that misbehave.

Disk usage — Checking how much storage is free and what is consuming it.

Package installation — Installing and updating software through the system package manager.

KEY COMMANDS

```
ls · cd · pwd
```

List contents · change directory · print current directory.

```
mkdir · rm · cp · mv
```

Make a folder · remove · copy · move or rename.

```
chmod · chown
```

Change file permissions · change file ownership.

```
ps aux · kill -9
```

List all processes · force-stop a process by its ID.

```
df -h · du -sh
```

Show free disk space · show a folder's total size.

```
sudo apt update
```

Refresh the package lists on Debian/Ubuntu systems.

14. Project Setup from GitHub

The standard sequence for taking a cloned repository from download to a running application.

Clone the project — Download the repository to your machine to start working with it.

Read the README — The project's own setup and usage guide — always the first thing to read.

Create the environment — Make an isolated Python or Conda environment for the project's dependencies.

Install dependencies — Install the required packages, usually from requirements.txt or a lock file.

Set environment variables — Provide the API keys and config values the project expects, often via a .env file.

Download models / assets — Fetch any model weights or data files the project needs but doesn't include.

Run backend / frontend — Start the server side and the user-facing side of the application.

Test the APIs — Hit the endpoints to confirm the app runs and responds correctly.

15. Basic Troubleshooting Skills

A disciplined way to read what's wrong and check the usual suspects before asking for help.

Read the error message — The message and stack trace usually name the real problem; read it fully before guessing.

Search the error trace — Copy the key line into a search engine or docs — most errors have been seen before.

Check package versions — Confirm installed versions match what the project expects, since mismatches cause many failures.

Check GPU availability — Verify the GPU is visible (nvidia-smi, torch.cuda.is_available) when GPU code fails.

Check port conflicts — Make sure the port your app wants isn't already in use by another process.

Check system libraries and paths — Confirm required native libraries exist and that file paths and PATH entries are correct.

Check permissions and logs — Verify you have access to the files involved, and read the logs before escalating.

A note on using this handbook

These definitions are deliberately basic — enough to recognize a term and know where it fits. The next step for any item is hands-on practice: install it, run the verification command, and deliberately break and fix it once. Understanding sticks far better after you've seen both the working state and the error it throws.